Tema 2:
Programació multifil

Setmana 5. Sincronització amb wait i notify

Model Productor-Consumidor

Problema clàssic sincronització.

Fil produeix, fil consumeix.

Problema → Consumidor consumeix més ràpid que produeix el Productor.

Consumidor se salta dades.

Exemples → Escriptura en fitxer, eixida impressora, ...

Exemple Productor-Consumidor

Programarem, creant 3 classes:

- Cua (objecte compartit)
- Productor (inserirà en cua o put)
- Consumidor (extraurà de cua o get)

Productor

Fil Productor genera números del 0 al 4, els posa en l'objecte cua, visualitza acció i fa pausa.

```
3  class Productor extends Thread {
4      private Cua cua;
5      private int n;
6
7      public Productor(Cua c, int n) {
8          cua = c;
9          this.n = n;
10     }
11
12     public void run() {
13         for (int i = 0; i < 5; i++) {
14             cua.put(i); // posa el número
15             try {
16                 sleep(1000);
17             } catch (InterruptedException e) {
18             }
19         }
20     }
21 }
```

Consumidor

Similar a productor, només que recull de Cua cridant a get

Per a que el consumidor siga més ràpid que el productor, no fem pausa.

```
3  class Consumidor extends Thread {
4      private Cua cua;
5      private int n;
6
7      public Consumidor(Cua c, int n) {
8          cua = c;
9          this.n = n;
10     }
11
12     public void run() {
13         int valor = 0;
14         for (int i = 0; i < 5; i++) {
15             valor = cua.get();// recull el número
16         }
17     }
18 }
```

Cua

```
3  public class Cua {
4      // atributs
5      private int numero;
6      private boolean disponible = false; // inicialment buida
7
8      // mètodes
9      // traure
10     public int get() {
11         if (disponible) { // si no està buida
12             disponible = false; // pose que està buida
13             System.out.println("Consumeix: " + numero);
14             return numero; // torne numero
15         }
16         else {
17             System.out.println("Intenta consumir pero no consumeix");
18             return -1; // no hi ha numero
19         }
20     }
21
22     public void put(int valor) {
23         numero = valor; // col·locar valor a la cua
24         disponible = true; // disponible per a consumir, cua plena
25         System.out.println("Produeix: " + valor);
26     }
27
28 }
```

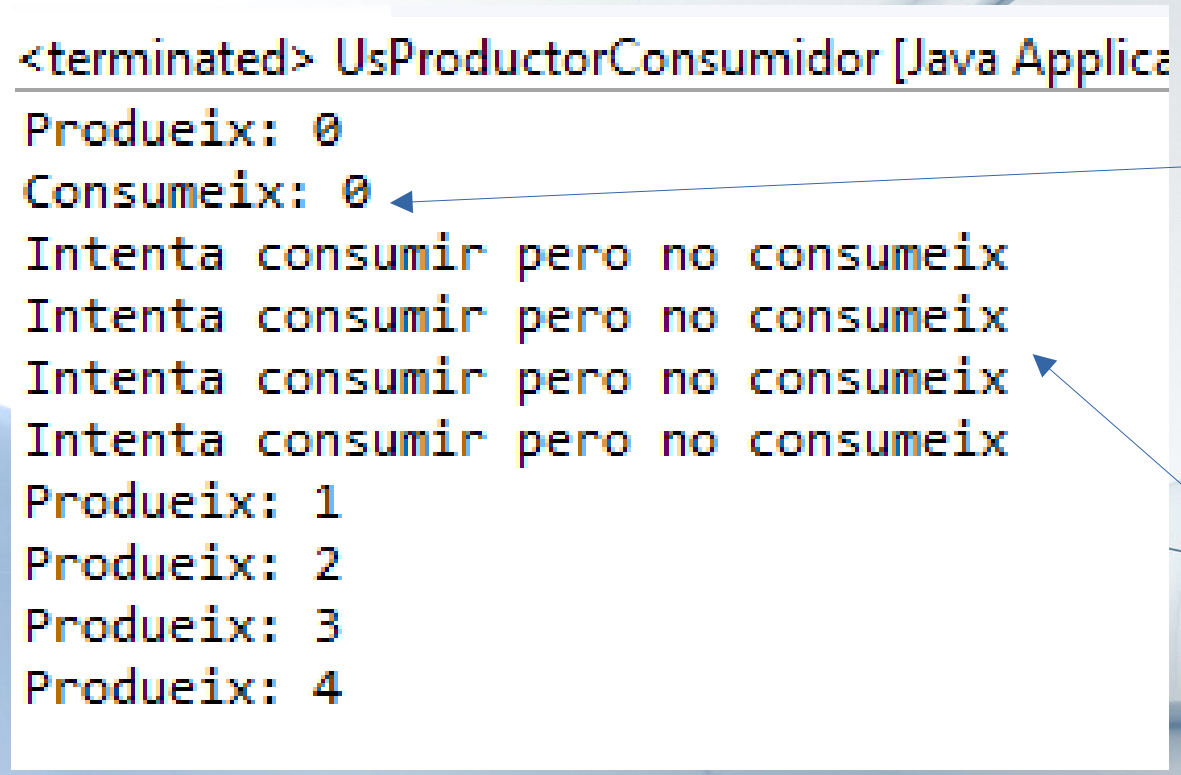
Mètode main

Crea objecte comú cua.

Crea Productor i Consumidor i els passa la cua

```
3  public class UsProductorConsumidor {
4      public static void main(String[] args) {
5          Cua cua = new Cua();
6          Productor p = new Productor(cua, 1);
7          Consumidor c = new Consumidor(cua, 1);
8          p.start();
9          c.start();
10     }
11 }
```

Execució



Després de produir, el productor fa pausa.

El consumidor consumeix una vegada.

Les altres 4 no pot consumir pq el productor està en pausa.

Eixida desitjada

```
<terminated> UsProducto
Produeix: 0
Consumeix: 0
Produeix: 1
Consumeix: 1
Produeix: 2
Consumeix: 2
Produeix: 3
Consumeix: 3
Produeix: 4
Consumeix: 4
```

Synchronized

Mètodes de classe Cua com synchronized.
Productor no pot canviar el valor de la cua quan el Consumidor està recollint valor,
Consumidor no pot recollir quan el Productor està canviant-la.

```
public synchronized int get() {}
public synchronized void put(int valor) {}
```

Coordinació

Quan Productor pose número en cua, que avise al Consumidor.
Quan la cua estiga disponible, que avise al Productor.

Per a aconseguir esta coordinació necessitem mètodes:
wait(), notify(), notifyAll()

Mètodes

- wait() → un fil que crida al mètode wait() d'un cert objecte queda suspés fins que un altre fil cride al mètode notify() o notifyAll() del mateix objecte. Si està dins d'un bloc synchronized, **allibera el bloqueig** (sleep no ho fa).

- notify() → desperta només a un dels fils que va realitzar una crida a wait(). Si varios fils estan esperant només un és escollit

- notifyAll() → desperta tots els fils que estan esperant l'objecte.

Mètode get

Ha d'esperar que la cua s'òmpliga. Mentre disponible siga false, espera (wait).

Ix del bucle quan arriba un valor, posa disponible a false i avisa a tots els fils que comparteixen la cua.

Mètode get

```
10      public synchronized int get() {
11          while (disponible == false) { // si està buida
12              try {
13                  wait();
14              } catch (InterruptedException e) {e.printStackTrace();};
15          }
16          disponible = false;
17          notifyAll();
18          System.out.println("Consumeix: " + numero);
19          return numero; // no hi ha numero
20      }
```

Mètode put

Ha d'esperar que la cua es buide. Mentre disponible siga true, espera (wait).
Ix del bucle, posa el valor a la cua, posa disponible a true i avisa a tots els fils que comparteixen la cua.

Mètode put

```
22⊖     public synchronized void put(int valor) {
23          while (disponible == true) { // si no està buida
24              try {
25                  wait();
26              } catch (InterruptedException e) {e.printStackTrace();}; };
27          }
28          numero = valor;
29          disponible = true;
30          System.out.println("Produeix: " + valor);
31          notifyAll();
32      }
33  }
```